REPORT

Reviewer report: python-attrs/attrs#1654: fix(validators):

name the failing inner item in

deep_iterable/deep_mapping errors

Run attrs-1654, tested 2026-11-09 11:21 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-09 15:04 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

COULD NOT VERIFY

Recommended action: Re-run Confidence: Low

The run did not produce a usable comparison, so there is no conclusion about the PR. The run did not finish

cleanly, so it says nothing about the PR.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: fix(validators): name the failing inner item in deep_iterable/deep_mapping errors

(python-attrs/attrs#1654).

• What we observed: unverified behaviour claim: "| format | ruff format --exit-non-zero-on-format . | left

unchanged — exit 0 |"; needs a probe (3 test(s) fail on main and pass on the PR; 170 test(s) ran on main, 170 on

the PR; 0 new test(s) pass, 167 unchanged; probe model-p1 is not_adequate (the probe crashed or printed no

parsed output on every main-side run; not counted as ran or adequate); its matching results do not count; probe

model-p2 is not_adequate (the probe crashed or printed no parsed output on every main-side run; not counted

as ran or adequate); its matching results do not count; probe model-p3 is not_adequate (the probe crashed or

printed no parsed output on every main-side run; not counted as ran or adequate); its matching results do not

count; probe model-p4 is not_adequate (the probe crashed or printed no parsed output on every main-side run;

not counted as ran or adequate); its matching results do not count).

• What you should do: Do not decide yet. Fix the problem under 'How to fix' and run the check again.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 5.6 GB (of 15.6 GB).

• Setup check: free disk 92.8 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: attrs python setup install instructions https://github.com/python-attrs/attrs

site:github.com/python-attrs/attrs

• Query: attrs python documentation install https://github.com/python-attrs/attrs site:github.com/python-attrs/attrs

• Query: attrs python install requirements https://pypi.org/project/attrs/ https://github.com/python-attrs/attrs

site:pypi.org

• Source: attrs/pyproject.toml at main · python-attrs/attrs · GitHub —

https://github.com/python-attrs/attrs/blob/main/pyproject.toml

• Source: attrs/README.md at main · python-attrs/attrs · GitHub —

https://github.com/python-attrs/attrs/blob/main/README.md

• Source: attrs — https://pypi.org/project/attrs/

• Source: Minor docs fixes · python-attrs/attrs@307a646 · GitHub —

https://github.com/python-attrs/attrs/commit/307a6467d37308c22af32605f7e1e5e2b3e452f1

• Source: attrs/docs/conf.py at main · python-attrs/attrs · GitHub —

https://github.com/python-attrs/attrs/blob/main/docs/conf.py

• Source: GitHub - python-attrs/attrs: Python Classes Without Boilerplate · GitHub —

https://github.com/python-attrs/attrs/tree/main

• Setup plan: the same packages on main and pr.

• Setup plan: install cloudpickle; platform_python_implementation == "CPython" (listed in the tests extra) on main

and pr.

• Setup plan: install hypothesis (listed in the tests extra) on main and pr.

• Setup plan: install pympler (listed in the tests extra) on main and pr.

• Setup plan: install pytest>9 (listed in the tests extra) on main and pr.

• Setup plan: install pytest-xdist[psutil] (listed in the tests extra) on main and pr.

• Setup plan: install watchfiles (listed in tox.ini deps) on main and pr.

• Setup plan: install cogapp (listed in tox.ini deps) on main and pr.

• Setup plan: install doc2dash (listed in tox.ini deps) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.10, >=3.9

• Installed cloudpickle 3.1.2 from pyproject (listed in the tests extra) on main and pr.

• Installed hypothesis 6.170.15 from pyproject (listed in the tests extra) on main and pr.

• Installed pympler 1.1 from pyproject (listed in the tests extra) on main and pr.

• Installed pytest 9.2.0 from pyproject (listed in the tests extra) on main and pr.

• Installed pytest-xdist 3.8.0 from pyproject (listed in the tests extra) on main and pr.

• Installed watchfiles 1.3.0 from tox.ini (listed in tox.ini deps) on main and pr.

• Installed cogapp 3.6.0 from tox.ini (listed in tox.ini deps) on main and pr.

• Installed doc2dash 3.1.0 from tox.ini (listed in tox.ini deps) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (167 unchanged, 3 runs per side) | pytest-pr-1.log:9 «170 passed in 1.33s»

| P-C1 | After the fix, a failing inner validator in deep_iterable produces an error message that includes the index

of the failing item, e.g., 'Length of 'x[1]' must be >= 1: 0'. (PR description) | not testable | probe model-p1 has no

observation for this claim | no matching log line

| P-C2 | After the fix, a failing inner validator in deep_mapping for a value produces an error message that

includes the index of the failing item, e.g., "'m[2]' must be <class 'str'> (got 2 that is a <class 'int'>)". (PR

description) | not testable | probe model-p2 has no observation for this claim | no matching log line

| P-C3 | After the fix, a failing inner validator in deep_mapping for a key produces an error message that includes

the key representation, e.g., "m['a']". (PR description) | not testable | probe model-p3 has no observation for this

claim | no matching log line

| P-C4 | On the base commit the four new tests for inner validator error messages all fail, while on the branch

they all pass. (PR description) | not testable | probe model-p4 has no observation for this claim | no matching log

line

Citation check (WARN): 1 citation(s) verified; 4 statement(s) have no matching log line.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 170 / 170 | | Tests unchanged | 167 | | New tests that pass | 0 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 3 |

tests.test_validators.TestDeepIterableMemberErrorMessage::test_instance_of_names_the_failing_member,

tests.test_validators.TestDeepIterableMemberErrorMessage::test_min_len_names_the_failing_member,

tests.test_validators.TestNot_::test_bad_exception_args

| Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-p1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p1.py, frozen c234753e70da

at 2026-11-09T05:53:18.863727+00:00): C1: deep_iterable inner validator error message

• P-C1 (PR description): After the fix, a failing inner validator in deep_iterable produces an error message that

includes the index of the failing item, e.g., 'Length of 'x[1]' must be >= 1: 0'. Expects the covered observations to

differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| probe.error | - | probe could not run | same on both sides: {"crash": "non-json output", "exit_code": 0} |

probe-model-p1-main-1.jsonl:1 «"observation": "probe.error"»
probe-model-p1-pr-1.jsonl:1 «"observation":

"probe.error"»

model-p2 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p2.py, frozen ff566d138c1c at

2026-11-09T05:53:18.863928+00:00): C2: deep_mapping value inner validator error message

• P-C2 (PR description): After the fix, a failing inner validator in deep_mapping for a value produces an error

message that includes the index of the failing item, e.g., "'m[2]' must be <class 'str'> (got 2 that is a <class 'int'>)".

Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| probe.error | - | probe could not run | same on both sides: {"crash": "non-json output", "exit_code": 0} |

probe-model-p2-main-1.jsonl:1 «"observation": "probe.error"»
probe-model-p2-pr-1.jsonl:1 «"observation":

"probe.error"»

model-p3 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p3.py, frozen c13bd4143b65

at 2026-11-09T05:53:18.864024+00:00): C3: deep_mapping key inner validator error message

• P-C3 (PR description): After the fix, a failing inner validator in deep_mapping for a key produces an error

message that includes the key representation, e.g., "m['a']". Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| probe.error | - | probe could not run | same on both sides: {"crash": "non-json output", "exit_code": 0} |

probe-model-p3-main-1.jsonl:1 «"observation": "probe.error"»
probe-model-p3-pr-1.jsonl:1 «"observation":

"probe.error"»

model-p4 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p4.py, frozen 167edfea5e8f at

2026-11-09T05:53:18.864130+00:00): C4: test pass count for inner validator error messages

• P-C4 (PR description): On the base commit the four new tests for inner validator error messages all fail, while

on the branch they all pass. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| probe.error | - | probe could not run | same on both sides: {"crash": "non-json output", "exit_code": 0} |

probe-model-p4-main-1.jsonl:1 «"observation": "probe.error"»
probe-model-p4-pr-1.jsonl:1 «"observation":

"probe.error"»

diff-api (diff (auto-derived from base-side code, model-free); informational under baseline full,

diff-probes/diff-api.py, frozen 889bf472b523 at 2026-11-09T05:53:18.867259+00:00): Public functions touched

by the PR, called with type-appropriate inputs on both sides.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| attr.validators.is_callable.input0 | - | same on both sides | same on both sides: {"repr": "<is_callable validator>",

"type": "_IsCallableValidator"} | probe-diff-api-main-1.jsonl:1 «"observation":

"attr.validators.is_callable.input0"»
probe-diff-api-pr-1.jsonl:1 «"observation":

"attr.validators.is_callable.input0"»

Citation check (PASS): 10 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (4.0 s run).

Coverage gate (python, confidence normal): 12 of 14 changed executable line(s) ran (86%).

| Changed file | Changed executable lines | Of those, ran

| src/attr/validators.py | 14 | 12

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 05e225347d4c of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 12 of 14 changed executable line(s) ran (86%).

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every probe observation cites its output line: 10 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Run did not finish | COULD NOT VERIFY | unverified behaviour claim: "| format | ruff format

--exit-non-zero-on-format . | left unchanged — exit 0 |"; needs a probe (3 test(s) fail on main and pass on the PR;

170 test(s) ran on main, 170 on the PR; 0 new test(s) pass, 167 unchanged; probe model-p1 is not_adequate

(the probe crashed or printed no parsed output on every main-side run; not counted as ran or adequate); its

matching results do not count; probe model-p2 is not_adequate (the probe crashed or printed no parsed output

on every main-side run; not counted as ran or adequate); its matching results do not count; probe model-p3 is

not_adequate (the probe crashed or printed no parsed output on every main-side run; not counted as ran or

adequate); its matching results do not count; probe model-p4 is not_adequate (the probe crashed or printed no

parsed output on every main-side run; not counted as ran or adequate); its matching results do not count) | Read

the setup/build log under Evidence links; build failures usually need a compiler or a newer nightly.

| Probes ran on both sides | WARN | model-p1 main#1; model-p1 pr#1; model-p1 main#2 | Read the probe-*.log

files; a probe that cannot run is not evidence either way.

| Every claim cites a real log line | WARN | 1 citation(s) verified; 4 statement(s) have no matching log line |

Regenerate the report; if it persists, the logs were edited or truncated.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 12 of 14 changed executable line(s) ran (86%)

| Probes ran on both sides | WARN | model-p1 main#1; model-p1 pr#1; model-p1 main#2

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | WARN | 1 citation(s) verified; 4 statement(s) have no matching log line

| Every probe observation cites its output line | PASS | 10 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-09/attrs-1654 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/python-attrs/attrs/pull/1654

Evidence links

• Pull request python-attrs/attrs#1654

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-model-p1-main-1.jsonl

• probe-model-p1-pr-1.jsonl

• probe-model-p2-main-1.jsonl

• probe-model-p2-pr-1.jsonl

• probe-model-p3-main-1.jsonl

• probe-model-p3-pr-1.jsonl

• probe-model-p4-main-1.jsonl

• probe-model-p4-pr-1.jsonl

• probe-diff-api-main-1.jsonl

• probe-diff-api-pr-1.jsonl

• main run 1 result: pytest-main-1.log:9 «168 passed in 1.80s»

• pr run 1 result: pytest-pr-1.log:9 «170 passed in 1.33s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-09 18:39 IST